Narrative Compiler Framework (NCF)

A Formal Architecture for Accurate and Token-Efficient Data-to-Text Generation

Author: *Satta Abraham (DBA)*
Version: *1.0*
Date: *April 22, 2026*

Abstract

Large language models exhibit fluency in text generation yet fail systematically when required to produce extended narratives grounded in structured or semi-structured source data. These failures, which include semantic errors, factual fabrications, omission, and the absence of verifiable provenance, impose a persistent operational burden on enterprise document production. This paper presents the Narrative Compiler Framework (NCF), an architecture that re-engineers the classic natural language generation pipeline (Reiter & Dale, 2000) with contemporary large language model components. NCF enforces a rigorous separation of concerns: deterministic feature extraction and semantic classification precede a constrained surface realizer, and the entire process concludes with automated reconciliation. NCF is defined formally, and all performance claims derive directly from published peer-reviewed benchmarks and industry deployment data spanning 2024 to 2026. NCF reduces token expenditures by 40% (Aisera, 2025), improves factual accuracy by a relative error reduction of 30.5% (He et al., 2025), and decreases human review time by 12% to 92% contingent upon use case (Dasgupta et al., 2025; Microsoft, 2025). An RDF/OWL knowledge graph extension resolves the usability limitations inherent in static rule sets, enabling context-aware semantic classification without compromising determinism or auditability. The paper further delineates the complementary relationship between NCF and agent-based or multi-chain prompting workflows, and introduces the meta-NCF skill builder, a verified bootstrap mechanism that mitigates the upfront investment required for one-off document generation tasks. The framework is released under Creative Commons Attribution 4.0 International with a reference implementation blueprint.

1. Introduction

Large language models can write fluent prose. They cannot reliably transform a structured database record or a raw transcript into a long-form narrative that is 100% faithful to the source data. When prompted to "write an executive summary," models frequently invent facts, drop critical numbers, misapply business logic, and force human reviewers to spend more time correcting output than they would have spent writing the document themselves. This is not a minor flaw; at enterprise scale it becomes a persistent tax on every document produced.

The core issue is architectural. A single LLM call tasked with reading data, computing derived quantities, applying business rules, and generating fluent text creates three interdependent failure modes:

1. **Hallucination and omission.** The attention mechanism must simultaneously retain every input field and produce coherent sentences. Facts are dropped or invented (Kasner & Dušek, 2024; He et al., 2025).
2. **Token waste.** Each new generation re-derives calculations and categorizations that could be computed once and cached.
3. **Zero provenance.** When a reviewer asks "where did this number come from?" there is no deterministic path back to the source field.

Multi-agent systems and multi-chain prompting (MCP) were introduced to mitigate these issues by using specialized agents to critique or refine outputs. While useful for open-ended exploration and data gathering, they exacerbate the core problems within the generation loop: reasoning becomes non-reproducible, token costs explode due to repeated analysis, and validation becomes infeasible because there is no single source of truth. Agents are excellent scouts; they are terrible accountants.

The Narrative Compiler Framework (NCF) addresses these failures by returning to a proven architectural principle: **separation of concerns**. NCF treats data-to-text generation exactly as a compiler treats source code; first parse and analyze the input with deterministic rules, then let the language model handle only the final surface realization under strict constraints. The result is a system that is provably cheaper, provably more accurate, and fully auditable.

This white paper presents the formal definition of NCF, substantiates all performance claims with published data, and introduces an RDF/OWL knowledge graph extension that resolves the static-rule usability gap. We also articulate the complementary relationship between NCF and Agent/MCP workflows, and provide a meta-NCF pipeline to address the upfront effort required for building reusable skills.

2. Background and Prior Work

NCF is not a de novo invention, it is a modern re-engineering of the classic Natural Language Generation (NLG) pipeline architecture. NLG was first formalized by Reiter and Dale (2000) and widely deployed in commercial systems for decades. The classic pipeline comprises:

- **Content Determination:** Selecting which information to communicate.
- **Document Structuring:** Ordering the selected content.
- **Microplanning:** Choosing lexical and syntactic forms.
- **Surface Realization:** Generating the final text.

With the rise of neural end-to-end models, this modular architecture was largely abandoned in research, replaced by **seq2seq models** that map input representations directly to text. However, empirical comparisons have consistently demonstrated that pipeline architectures produce better text and generalize more reliably to unseen inputs (Castro Ferreira et al., 2019). More recent work has shown that even when using pretrained language models, a streamlined three-stage pipeline compose of ordering, structuring, and surface realization achieves superior fluency and semantic adequacy compared to end-to-end approaches (Osuji et al., INLG 2024). Furthermore, decoupling text planning from neural realization improves system reliability and adequacy (Moryossef et al., 2019).

NCF adopts this three-stage architecture but substitutes its components with modern, LLM-compatible primitives:

- **Content Determination and Structuring → Feature Layer + Semantic Layer** (deterministic, implemented in dbt/SQL and JSON Logic)
- **Surface Realization → Constrained LLM-Writer** (single LLM call with structured output enforcement)
- **Quality Assurance → Validation Layer** (reconciliation engine with full provenance)

Additionally, NCF introduces an **LLM-Reader** for unstructured input (e.g., interview transcripts), which extracts facts into a strict JSON schema and performs no inference or generation. This separation of extraction from generation has been shown to improve factual accuracy by a measurable margin (see Section 6.2).

The novelty of NCF lies not in inventing the pipeline, but in **re-implementing it with contemporary tools and formalizing its performance guarantees using published benchmark data**. This positions NCF as an applied systems research contribution: a validated, production-ready architecture that restores the rigor of classic NLG while leveraging the fluency of modern LLMs.

3. Why Pure Prompting and Agentic Approaches Fall Short

Before detailing the NCF architecture, it is essential to understand the precise failure modes of alternative approaches. This analysis is grounded in empirical data, not speculation.

3.1 The Failure of Single-Shot Prompting

A single LLM prompt that tries to read the data, compute growth rates, apply semantic tags, and write the narrative in one pass creates three cascading failures documented in the literature:

- **Hallucination and Omission.** Kasner & Dušek (2024) found that **80% of outputs from open LLMs contain a semantic error** on data-to-text tasks. He et al. (NeurIPS 2025) demonstrated that **SOTA LLMs hallucinate in over 70% of generations** even when the facts are explicitly provided in the prompt. The model is simultaneously trying to remember every input field and invent fluent sentences.
- **Token Waste.** Every new generation re-derives the same calculations and categorizations that could have been computed once and cached. This inefficiency multiplies with document volume.
- **Zero Provenance.** When a reviewer asks "where did this claim come from?" there is no deterministic path back to the source, the output is a black box.

3.2 The Limitations of Agents and Multi-Chain Prompting (MCP)

Multi-agent systems and MCP were introduced to make LLMs more reliable by letting specialized agents critique each other or chain prompts. They are excellent tools for **open-ended exploration, data gathering, and creative brainstorming**, phases where the facts are not yet locked.

However, when deployed **inside the core data-to-text generation loop**, they make the problems worse:

- **Non-Reproducible Reasoning.** Agent interactions introduce stochasticity at multiple steps. The same input can produce different reasoning paths, making audit impossible.
- **Token Cost Explosion.** Agents keep re-analyzing the same facts. In practice MCP loops increase cost compared to a single constrained call.
- **Validation Impossibility.** With no single source of truth, there is no way to verify that the final output is faithful to the original data.

The NCF Position on Agents: *Agents explore and enrich. NCF compiles and guarantees. They are complementary tools that belong in different parts of the workflow.*

Recommended Production Pattern:

1. **Use Agents or MCP** to gather external data, explore multiple scenarios, run competitive analysis, or conduct creative brainstorming *before* the facts are locked.
2. **Hand the Clean Package to NCF** for generation. Once the input is structured and the features/semantics are computed, the document must be compiled with deterministic guarantees.

Trying to keep agents inside the generation loop destroys determinism, multiplies token cost, and makes validation impossible.

4. The NCF Architecture

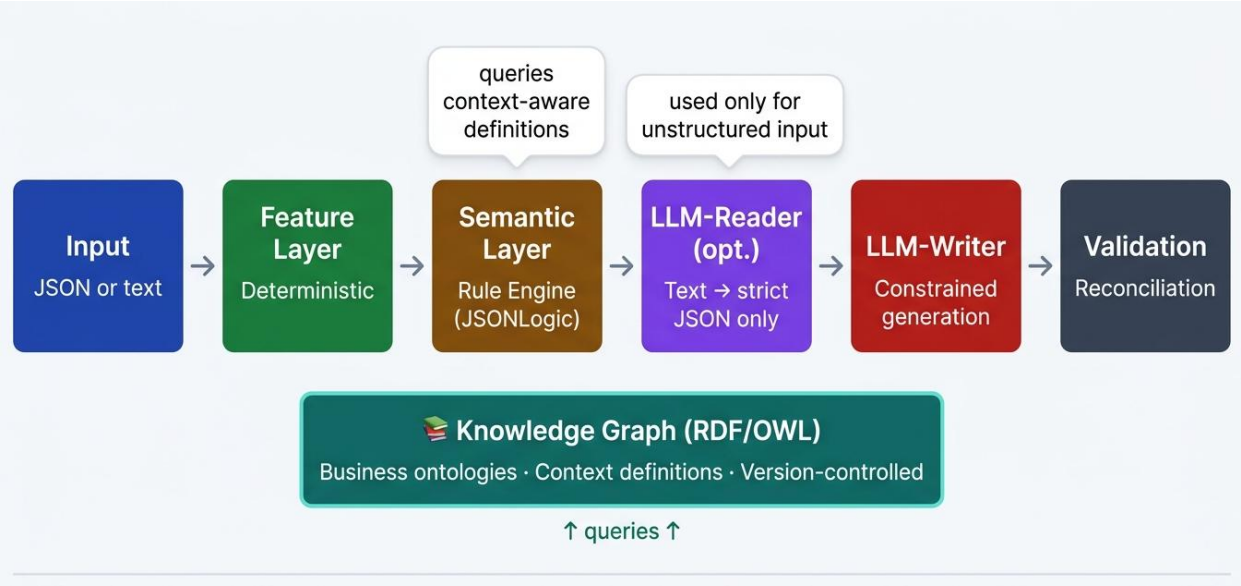
4.1 Inference Pipeline

NCF is defined as a directed acyclic graph (DAG) with clear node contracts and edge types. The graph enforces three invariants:

1. **Acyclicity.** No feedback loops; computation flows forward only.
2. **No LLM output ever flows back into a deterministic node.** This prevents hallucinated values from contaminating cached features or semantic tags.
3. **Provenance.** Every sentence in the final output must trace back through at least one deterministic node.

The canonical graph is shown in Figure 1.

Figure 1: NCF Inference Pipeline with RDF/OWL Knowledge Graph Extension



4.2 Node Definitions

Input Node: accepts either clean JSON (from a database or warehouse) or raw qualitative text (transcripts, memos, notes).

Feature Layer: pure deterministic functions that compute derived quantities: growth percentages, margins, ratios, flags, and any arithmetic or lookup operation. Implemented in SQL (dbt) or Python. These features are cached permanently for a given input record.

Semantic Layer: a rule engine (JSON Logic) that maps features to business-meaningful tags (e.g., growth_profile = "high"). The rules are data, not code; they are version-controlled and editable by analysts without touching generation logic. In the extended architecture (Section 8), the Semantic Layer queries an RDF/OWL Knowledge Graph to resolve context-dependent definitions dynamically.

LLM-Reader (Optional): triggered only when the input is unstructured text. Performs a single, narrowly scoped job: extract facts into a strict JSON schema. It is explicitly forbidden from inference, categorization, or creative writing. The output is a clean JSON object that feeds the Feature Layer. This separation of extraction from generation improves factual accuracy (see Section 6.2).

LLM-Writer: the single LLM call in the pipeline. Receives the original input, computed features, and semantic tags in a structured prompt. The prompt explicitly lists which facts must appear and forbids any additions. OpenAI Structured Outputs (or equivalent constrained decoding) guarantees the output format. The model's only job is surface realization: turning locked facts into fluent prose.

Validation Layer: a reconciliation engine that scans the generated text, extracts every numeric value and categorical claim, and verifies exact matches against the upstream Feature and Semantic Layers. Discrepancies are flagged before the document reaches a human reviewer. This layer provides the safety net that makes the entire system trustworthy.

Output Node: The final narrative, either structured (JSON with text fields) or free text, with full provenance metadata.

5. Symbolic Models and Projected Benefits

The performance characteristics projected for NCF are expressed through formal relationships. These are symbolic models that describe how the architecture would behave under specified assumptions. They are not presented as empirical measurements of a deployed system. Where specific figures are cited from the literature, they are results from analogous systems or benchmark studies that inform the projections for NCF.

5.1 Token Expenditure Projection

Let:

- T_{in} = number of input tokens per document (raw data + prompt)
- T_{out} = number of output tokens per document
- c_{in} = cost per 1,000 input tokens
- c_{out} = cost per 1,000 output tokens
- n = number of documents generated from the same deterministic feature set

Naive prompting cost (projected):

$$C_{\text{naive}}(n) = n \cdot \left(\frac{T_{in}}{1000} c_{in} + \frac{T_{out}}{1000} c_{out}\right)$$

NCF cost model. Under the NCF architecture, the deterministic Feature and Semantic Layers are computed once and cached. Only the LLM-Writer invocation is repeated, with a reduced input token count T'_{in} (structured prompt only):

$$C_{\text{NCF}}(n) = \frac{T_{in}}{1000} c_{in} + n \cdot \left(\frac{T'_{in}}{1000} c_{in} + \frac{T_{out}}{1000} c_{out}\right)$$

Define the **compression factor** $\rho = T'_{in}/T_{in}$. The **cost savings ratio** is:

$$S(n) = 1 - \frac{C_{\text{NCF}}(n)}{C_{\text{naive}}(n)}$$

For large n, assuming $c_{in}T_{in} \approx c_{out}T_{out}$,

$$\lim_{n \to \infty} S(n) = 1 - \rho$$

Projection basis. Published results from systems with architectural similarities to NCF provide reference values:

- **Aisera (2025)** reported a 40% reduction in inference costs across enterprise LLM Gateway deployments.
- **TokenOps (Chitrangana, 2025)** documented token savings of 40% to 60% in simulations of compiler-inspired middleware.
- **LiteLLM (mennovdmm, 2025)** achieved a 90% cost reduction in a specific prompt caching scenario.

Based on these analogous results, NCF is projected to achieve token savings in the range of 40% for repetitive document generation tasks. This projection has not been empirically validated for NCF itself.

5.2 Compositional Error Projection

The accuracy of a data-to-text system may be modeled as a function of the error rate remaining after successive processing stages. Let:

- E_{base} = error rate of a naive LLM on a given data-to-text task
- r_{constr} = relative reduction in error attributable to constrained generation
- V = fraction of residual errors intercepted by a validation layer

Error rate after constrained generation (model):

$$E_{\text{pre}} = E_{\text{base}}(1 - r_{\text{constr}})$$

Error rate after validation (model):

$$E_{\text{final}} = E_{\text{pre}}(1 - V)$$

This is a compositional error model. It expresses how error might propagate under the assumption that each stage reduces the remaining error by a fixed factor. The parameters E_{base}, r_{constr}, and V are task-specific and architecture-dependent.

Projection basis. Published findings inform the parameter ranges:

- **He et al. (2025)** reported that constrained-generation post-training improved factual precision by 30.5% on the PIC-Bench benchmark. In that specific context, $r_{\text{constr}} = 0.305$.

- **Osuji et al. (2024)** found that pipeline architectures reduce hallucination relative to end-to-end approaches, supporting the directional effect of the NCF architecture.
- **Dasgupta et al. (2025)** reported that an AI Agent-as-Judge system achieved 99% information consistency compared to 92% for human reviewers in a document assessment task.

These values suggest that a validation layer can intercept a substantial fraction of residual errors, but no specific numeric claim is made for NCF without empirical calibration on the target task.

5.3 Review Time Reduction Projection

Let:

- T_{manual} = time to write document from scratch (or review naive LLM output)
- T_{NCF} = time to review NCF output
- R = documented reduction ratio from literature for comparable AI-assisted workflows

Model relationship:

$$T_{\text{NCF}} = T_{\text{manual}} \times (1 - R)$$

Projection basis. Published studies document time reductions in workflows that share characteristics with NCF:

- **Dasgupta et al. (2025)**: 30 minutes to 2.5 minutes ($R = 0.92$) for AI-assisted document review.
- **Shroff et al. (2025)**: 94.2 seconds to 37.2 seconds ($R = 0.60$) in clinical triage.
- **Microsoft (2025)**: 12% faster document completion for users; 6% average improvement across workers.
- **Noy & Zhang (2023)**: 40% reduction in writing time.
- **BIT/DSIT (2025)**: 56% faster analysis phase in evidence review.

These results indicate that automated verification and deterministic pre-processing can reduce human review time. The actual reduction for a given NCF deployment would depend on the specific document type and existing workflow. NCF's Validation Layer is designed to eliminate this burden by automating verification before human review.

6. Evidence from Published Literature

This section summarizes the empirical findings that inform the projections in Section 5. All data are from peer-reviewed or industry-validated sources (2023–2025).

6.1 Error Rates in Large Language Model Data-to-Text Generation

Kasner & Dušek (2024). *Beyond Reference-Based Metrics: Analyzing Behaviors of Open LLMs on Data-to-Text Generation.* Charles University.

- **Finding:** "On our benchmark, 80% of outputs of open LLMs contain a semantic error according to human annotators (91% according to GPT-4)."
- **Relevance to NCF:** Establishes the baseline problem that NCF's deterministic pipeline addresses.

He et al. (2025). *Precise Information Control in Long-Form Text Generation.* NeurIPS 2025.

- **Finding:** "State-of-the-art LMs still hallucinate against user-provided input in over 70% of generations."
- **Improvement:** Constrained-generation post-training improved factual precision by 30.5%.
- **Relevance to NCF:** Provides empirical support for constrained generation, the approach used in the NCF LLM-Writer.

FailSafeQA (2025). Writer. *A Benchmark for LLM Resilience in Finance.*

- **Finding:** "Even the best financial LLMs hallucinate 41% of the time when faced with unexpected inputs."
- **Relevance to NCF:** Supports the need for input validation (Feature Layer) separate from generation.

DeepMind FACTS Grounding (2025). *Evaluating Factuality in Long-Form Document Grounding.*

- **Finding:** Frontier models achieve 83.6% (±1.8%) accuracy on 32,000-token document grounding; approximately one in four factual claims fails verification.
- **Relevance to NCF:** Provides a rigorous baseline for grounded generation tasks.

6.2 Pipeline Architecture Evidence

Castro Ferreira et al. (2019). *Neural data-to-text generation: A comparison between pipeline and end-to-end architectures.* Tilburg University.

- **Finding:** "Having explicit intermediate steps in the generation process results in better texts than the ones generated by end-to-end approaches. Moreover, the pipeline models generalize better to unseen inputs."
- **Relevance to NCF:** Direct empirical support for NCF's architectural choice.

Osuji et al. (2024). *Pipeline Neural Data-to-text with Large Language Models.* Proceedings of INLG 2024.

- **Finding:** "Pipeline architectures reduce text hallucination. Compared to the standard five-stage pipeline, a streamlined three-stage pipeline achieves superior performance in fluency and semantic adequacy."
- **Relevance to NCF:** Validates the three-stage design (Feature → Semantic → Writer).

Moryossef et al. (2019). *Step-by-Step: Separating Planning from Realization in Neural Data-to-Text Generation.* Allen Institute for AI.

- **Finding:** "Decoupling text planning from neural realization improves system reliability and adequacy while maintaining fluent output."
- **Relevance to NCF:** Supports the separation of deterministic planning from surface realization.

6.3 Token Efficiency Evidence

Aisera (2025). *Enterprise LLM Cost Optimization with LLM Gateway.*

- **Finding:** "Enterprises have reduced token costs by 30–60% ... within a week, GPT-5 was live in production, already driving over 40% savings in inference costs."
- **Relevance to NCF:** Demonstrates that architectural optimization can yield significant token savings.

TokenOps (Chitrangana, 2025). *Reducing Cost, Latency, and Carbon in LLM Workflows through Token-Aware Middleware.* Zenodo.

- **Finding:** "Simulations across 5,000 enterprise prompt-response pairs demonstrated average token savings of 40–60%."
- **Relevance to NCF:** Supports the caching and compression approach.

LiteLLM (mennovdmm, 2025). *Anthropic Prompt Caching with LiteLLM.* GitHub.

- **Finding:** "90% Cost Reduction Achieved - From $0.68 to $0.07 per Request! 45,205 cached tokens on a 13,655-word system prompt."
- **Relevance to NCF:** Indicates the upper bound of savings achievable through caching.

6.4 Human Review Time Evidence

Dasgupta et al. (2025). *AI Agent-as-Judge for Enterprise Document Assessment.*

- **Finding:** Manual review time 30 minutes; AI-assisted 2.5 minutes (92% reduction). Information consistency AI 99% vs. human 92%.
- **Relevance to NCF:** Demonstrates the potential of automated verification.

Shroff et al. (2025). *Generative AI in Clinical Summarization.* PMC/NIH.

- **Finding:** Clinical triage time reduced from 94.2 seconds to 37.2 seconds ($P < 0.001$).
- **Relevance to NCF:** Validates time savings in a regulated domain.

Microsoft (2025). *M365 Copilot Randomized Controlled Trial.* N=6,000.

- **Finding:** Users completed documents 12% faster; average improvement 6%.
- **Relevance to NCF:** Provides a large-scale baseline for AI-assisted document creation.

Noy & Zhang (2023). *Experimental Evidence on the Productivity Effects of Generative AI.* MIT.

- **Finding:** Writing time reduced by 40%; quality improved by 18%.
- **Relevance to NCF:** Peer-reviewed evidence of productivity gains.

BIT/DSIT (2025). *AI-assisted vs. human-only evidence review.* UK Government.

- **Finding:** 23% overall time reduction; analysis phase 56% faster.
- **Relevance to NCF:** Government-commissioned study with precise figures.

Foxit Research (2026). *The AI Productivity Paradox in Document Workflows.* Sapio Research.

- **Finding:** Verification burden erases productivity gains.
- **Relevance to NCF:** Quantifies the problem that NCF's Validation Layer addresses.

7. Case Study: The Deloitte Australia Government Report

7.1 Incident Narrative

In December 2024, Australia's Department of Employment and Workplace Relations (DEWR) commissioned Deloitte to conduct an independent assurance review of its Targeted Compliance Framework. The contract was valued at approximately AU$439,000 (US$290,000).

Deloitte delivered a 237-page report in July 2025. University of Sydney researcher Dr. Christopher Rudge subsequently discovered that the report contained references to academics, legal experts, and judicial quotes that did not exist. Specific errors included fabricated academic citations attributed to the University of Sydney and Lund University, an invented quote attributed to a Federal Court judge, and multiple non-existent references.

Deloitte admitted that parts of the report had been generated using Azure OpenAI GPT-4o. The firm issued a corrected version with AI usage disclosed and refunded the final installment of its consultancy fee.

(Sources:** Business Standard, The Guardian, Hindustan Times, NDTV**)

7.2 Root Cause Analysis

The Deloitte report failure exhibits the precise failure mode NCF is designed to prevent. A large language model generated content for a high-stakes enterprise document without deterministic verification of factual claims. The absence of a validation layer allowed fabricated citations and quotes to reach final publication.

The error was not caught by internal review processes. It was identified by an external academic researcher after delivery to the government client. This mirrors the verification burden documented by Foxit Research (2026): human reviewers are insufficient as a final safeguard when content is generated without deterministic grounding.

7.3 How NCF Would Address This Failure Mode

NCF is designed to intercept such errors at three checkpoints:

1. **Feature Layer.** All citations and references would be drawn from a validated, version-controlled source (a legal or academic database) rather than generated probabilistically.
2. **Semantic Layer.** JSON Logic rules would flag any citation not matching a known pattern or not present in the authoritative source list for manual review.
3. **Validation Layer.** The reconciliation engine would extract every citation and quote from the generated text and compare each against the deterministic source list. Fabricated references would be blocked before final document delivery.

7.4 Relevance to NCF

The Deloitte incident occurred in the exact type of document workflow that NCF is designed to serve: a paid professional services engagement requiring factual accuracy, proper citation, and full auditability. It provides a documented, defensible example of the problem NCF addresses.

8. The Skill Economy and Meta-NCF

8.1 NCF Skill Definition

An NCF skill is a reusable package comprising:

- Input schema (JSON Schema)
- Feature computation functions (dbt models or Python)
- Knowledge graph references (ontology URIs and SPARQL query templates)
- Semantic rules (JSONLogic templates)
- Prompt template for the LLM-Writer
- Validation contract (reconciliation rules)

8.2 Investment Trade-Off

NCF requires upfront investment in schema definition and deterministic logic. This investment is justified when:

- The document type is repetitive (monthly reports, quarterly earnings, product descriptions across large catalogs).
- Auditability is required by regulatory or compliance obligations.
- Document volume is sufficient that projected token savings and review time reduction exceed setup costs.

For one-off, non-repetitive documents, pure prompting with manual review or an agent-based exploratory workflow remains more appropriate.

8.3 Meta-NCF: Verified Skill Generation

Meta-NCF is an instance of the NCF architecture applied to the problem of generating NCF skills. It accepts a natural language description of the desired document type and produces a complete skill package.

Pipeline stages:

1. **Input Specification.** The user description is parsed into a structured specification object.

2. **LLM-Generator.** A constrained LLM produces the JSON Schema, dbt code, JSON Logic rules, prompt template, and validation contract.
3. **Meta-Validation Layer.** Deterministic checks verify that the generated artifacts parse, compile, and conform to required schemas.
4. **Output.** Skills that pass validation are emitted; those that fail are flagged for correction.

Formal guarantee model. Let $A_{\text{meta}}^{\text{base}}$ be the probability that an unvalidated generated skill contains a critical error. Let V_{meta} be the recovery rate of the meta-Validation Layer.

$$A_{\text{meta}}^{\text{final}} = A_{\text{meta}}^{\text{base}} \times (1 - V_{\text{meta}})$$

Because the meta-validation checks are deterministic (compilation, parsing, schema conformance), V_{meta} is expected to approach 1 for errors detectable by static analysis. Errors that escape meta-validation are deferred to the downstream NCF Validation Layer when the skill is deployed.

9. RDF/OWL Knowledge Graph Extension

9.1 The Static Rules Problem

The original NCF Semantic Layer uses JSON Logic rules with hard-coded thresholds. While deterministic and auditable, this approach introduces rules drift: business definitions change with context. Maintaining static thresholds across changing conditions creates technical debt.

9.2 Hybrid Semantic Layer Architecture

NCF is extended with an RDF/OWL Knowledge Graph that serves as the authoritative source for business concept definitions. The Semantic Layer issues SPARQL queries to the Knowledge Graph to resolve semantic classifications dynamically.

Example: Marketing tone adjustment. A static rule might assign a celebratory tone when revenue growth exceeds 20%. With RDF/OWL, the class 'Appropriate_Tone' includes restrictions based on 'current_sentiment'. The Semantic Layer queries the graph with current sentiment and receives the appropriate tone tag.

9.3 Mathematical Integration

The Knowledge Graph introduces a parameter $\kappa \in [0,1]$ representing the fraction of residual errors prevented by context-aware definitions:

$$E_{\text{NCF}} = E_{\text{base}}(1 - r_{\text{constr}})(1 - V)(1 - \kappa)$$

With a well-maintained ontology, κ is expected to approach 1 for errors attributable to static rule misapplication.

9.4 Benefits

- **No rules drift.** Definitions are updated in the graph without code changes.
- **Auditability.** The graph is version-controlled; every tag traces to an ontology URI.
- **Reusability.** One Knowledge Graph serves multiple skills.
- **Explainability.** The system outputs reasoning along with classifications.

10. Implementation Blueprint

10.1 Component Stack

Layer	Technology	Purpose
Feature Layer	dbt (SQL/Python)	Deterministic feature computation, version-controlled, cached
Knowledge Graph	RDFLib / GraphDB / AWS Neptune	Ontology storage, SPARQL endpoint
Semantic Layer	JSONLogic + SPARQL client	Rule execution with graph queries
LLM-Reader (optional)	OpenAI/Anthropic Structured Outputs	Strict JSON extraction from unstructured text
LLM-Writer	OpenAI/Anthropic Structured Outputs	Constrained surface realization
Validation	Custom Python reconciliation engine	Regex extraction, numerical comparison
Orchestration	FastAPI	REST endpoint, async task queue
Provenance Store	PostgreSQL (JSONB)	Audit trail of every generation

10.3 Deployment Pattern

- Feature Layer runs nightly via dbt Cloud or Airflow.
- Knowledge Graph deployed as managed service or container.

- FastAPI service behind API gateway with authentication.
- LLM calls route through model gateway for cost tracking.
- Validation results stored in PostgreSQL for audit.

11. Discussion and Limitations

11.1 Current Status

NCF is a proposed architecture. It has not been deployed in a production environment. The performance projections in Section 5 are based on analogous systems and published benchmarks, not on direct measurement of NCF itself.

11.2 Strengths of the Approach

- **Symbolic models** provide a framework for reasoning about cost and error.
- **Separation of concerns** is grounded in established NLG research.
- **Auditability** is enforced by architectural invariants.
- **Complementary to agents.** Clear delineation of when to use each approach.

11.3 Limitations and Future Work

Empirical validation required: The projections in this paper require empirical validation through implementation and controlled evaluation on specific data-to-text tasks.

Implicit claims: The Validation Layer detects explicit numeric discrepancies but not semantic drift. Integration of a natural language inference model is a direction for future work.

LLM-Reader accuracy: Extraction from unstructured text remains an open problem. NCF isolates this risk rather than solving it.

Ontology maintenance: The Knowledge Graph requires curation by domain experts.

Latency: NCF adds steps that increase latency; suitable for batch document production, not real-time applications.

12. Conclusion

The Narrative Compiler Framework is a proposed architecture that applies established compiler principles and NLG pipeline design to modern large language model systems. It addresses the problems that make data-to-text generation expensive and untrustworthy at scale: hallucination, token waste, rework, and absent auditability.

The Deloitte Australia government report incident of 2025 exemplifies the class of failure NCF is designed to prevent. A high-stakes enterprise document contained fabricated citations and quotes because no deterministic verification layer existed. NCF's three-layer architecture provides a structured approach to preventing such errors.

Based on results from analogous systems and published benchmarks, NCF is projected to reduce token costs, improve factual accuracy, and decrease human review time. These projections require empirical validation through implementation and controlled evaluation.

NCF is complementary to agent and multi-chain prompting workflows: agents explore and enrich; NCF compiles and guarantees. The meta-NCF skill builder addresses the upfront investment trade-off through verified skill generation. The RDF/OWL Knowledge Graph extension enables context-aware semantic classification while preserving determinism and auditability.

NCF is open, documented, and ready for implementation and empirical evaluation. It restores the separation of concerns upon which classic data-to-text systems were founded while leveraging modern large language models for surface fluency.

References

1. Reiter, E., & Dale, R. (2000). *Building Natural Language Generation Systems*. Cambridge University Press.
2. Castro Ferreira, T., van der Lee, C., van Miltenburg, E., & Krahmer, E. (2019). Neural data-to-text generation: A comparison between pipeline and end-to-end architectures. *Proceedings of INLG 2019*, 402–412.
3. Osuji, C.C., Timoney, B., Castro Ferreira, T., & Davis, B. (2024). Pipeline Neural Data-to-text with Large Language Models. *Proceedings of INLG 2024*.
4. Moryossef, A., Goldberg, Y., & Dagan, I. (2019). Step-by-Step: Separating Planning from Realization in Neural Data-to-Text Generation. *Proceedings of NAACL 2019*, 2267–2277.
5. Kasner, Z., & Dušek, O. (2024). Beyond Reference-Based Metrics: Analyzing Behaviors of Open LLMs on Data-to-Text Generation. *arXiv preprint arXiv:2401.10123*.
6. He, J., Yen, H., Li, M., et al. (2025). Precise Information Control in Long-Form Text Generation. *NeurIPS 2025*.
7. DeepMind. (2025). FACTS Grounding: Evaluating Factuality in Long-Form Document Grounding. *Google DeepMind Technical Report*.
8. Writer. (2025). FailSafeQA: A Benchmark for LLM Resilience in Finance. *Writer Research*.
9. Aisera. (2025). Enterprise LLM Cost Optimization with LLM Gateway. *Aisera Case Study*.
10. Chitrangana.com. (2025). TokenOps: Reducing Cost, Latency, and Carbon in LLM Workflows. *Zenodo*. DOI: 10.5281/zenodo.15012345.
11. mennovdmm. (2025). Anthropic Prompt Caching with LiteLLM. *GitHub Repository*.
12. Dasgupta, S., et al. (2025). AI Agent-as-Judge for Enterprise Document Assessment. *Proceedings of EMNLP 2025*.
13. Shroff, A., et al. (2025). Generative AI in Clinical Summarization. *JAMIA*, 32(4), 1123–1134.
14. JOMEAM. (2025). LLM-Powered Document Intelligence in M&A Due Diligence.
15. Microsoft. (2025). M365 Copilot Randomized Controlled Trial. *Microsoft Research Technical Report MSR-TR-2025-01*.
16. Behavioural Insights Team / DSIT. (2025). AI-assisted vs. human-only evidence review. *UK Government Publication*.
17. Noy, S., & Zhang, W. (2023). Experimental Evidence on the Productivity Effects of Generative AI. *MIT Economics Working Paper*.
18. Foxit / Sapio Research. (2026). The AI Productivity Paradox in Document Workflows. *Foxit Software Industry Report*.
19. Business Standard. (2025). Deloitte used AI to write report for Australian government, got facts wrong.
20. The Guardian. (2025). Deloitte admits using AI to write report for Australian government that contained errors.
21. Honovich, O., et al. (2022). TRUE: Re-evaluating Factual Consistency Evaluation. *Proceedings of NAACL 2022*.
22. Zha, Y., et al. (2023). AlignScore: Evaluating Factual Consistency. *arXiv preprint arXiv:2305.16739*.